

ML Math Project

Overview:

Deep learning framework like a 'mini Pytorch' written from scratch in Python that implements a sparse weight algorithm that prunes (deletes) connections amongst neurons

User Stories

- **As a Developer**, I want to create a **Value** object that stores a scalar and its gradient so I can track derivatives across mathematical operations (addition, multiplication, etc.).
- **As a Researcher**, I want to implement the **Chain Rule** in a **.backward()** function so that I can automatically compute the gradient of a complex function without manual derivation.
- **As a User**, I want to define a **Layer** class that performs a matrix multiplication ($Y = WX + b$) using only NumPy to understand the linear algebra of a forward pass.
- **As a Data Scientist**, I want to implement **Weight Pruning** logic to set weights below a certain threshold to zero, simulating "Sparsity" (the BeyondML concept).
- **As a Visitor to the Web App**, I want to slide a "Pruning Percentage" bar and see a **Heatmap** of my weight matrix update in real-time to visualize how sparsity affects the model.
- **As a Visitor**, I want to see a **Histogram** of weights before and after my math is applied to verify that the distribution has shifted toward zero.

RoadMap

Week 1: Goal: Build the recursive autograd engine that handles the chain rule.

- **Day 1-2: The Value Class.**
 - ☒ Implement scalar variables that store data and gradients.
 - ☒ Overload Python operators (`__add__`, `__mul__`, `__pow__`).
 - ☐ Implement the **.backward()** function using **Topological Sort** to ensure gradients flow in the correct reverse order.
 - Understand the backpropagation logic
- **Day 3-4: The Neuron & Layer API.**
 - ☐ Build a **Neuron** class (weights + bias + activation).
 - ☐ Build a **Layer** class (a list of neurons).
 - ☐ Build an **MLP** (Multi-Layer Perceptron) class to chain layers.
- **Day 5: Testing & Verification.**
 - ☐ Write a test script that compares your manual gradients against a simple PyTorch script. If they don't match, your chain rule logic has a bug.

Week 2: The Tensor Engine & Sparsity (The Performance)

Goal: Move from scalars to NumPy matrices for speed and implement the "BeyondML" pruning.

- **Day 1-2: Vectorization.**
 - ☐ Rewrite your engine to use `numpy.array`.
 - ☐ Implement the **Linear Layer** using matrix multiplication ($Y = XW + b$).
 - ☐ Implement the backward pass for matrices (the "Transpose Swap").
- **Day 3-4: The Sparsity Algorithm.**
 - ☐ Create a `Pruner` class.
 - ☐ Write the logic for **Magnitude Pruning**: Sort weights by absolute value and create a binary mask for the top $X\%$.
 - ☐ **The "BeyondML" Logic**: Modify your forward pass so that weights are always multiplied by the mask: $W_{\text{eff}} = W \odot \text{Mask}$.
- **Day 5: Training on Real Data.**
 - ☐ Train your sparse model on a simple dataset (like MNIST or a non-linear "moons" dataset).
 - ☐ Plot the "Accuracy vs. Sparsity" curve.

Week 3: The Software Wrapper (The Portfolio Piece)

Goal: Turn your scripts into a professional library and API.

- **Day 1-2: Refactoring & Packaging.**
 - ☐ Organize your code into the folder structure we discussed (`/engine`, `/nn`, `/optim`).
 - ☐ Write a `requirements.txt` and a `README.md` that explains the math using LaTeX.
- **Day 3-4: The FastAPI Backend.**
 - ☐ Create an endpoint `/train` that accepts hyperparameters.
 - ☐ Create an endpoint `/predict` that takes an input and returns a prediction using the *sparse* model.
 - ☐ Create a `/metrics` endpoint that returns the current sparsity percentage and memory savings.
- **Day 5: Containerization.**
 - ☐ **SWE Task**: Write a `Dockerfile` for your backend.
 - ☐ Run it locally to ensure your NumPy-based engine performs well inside a container.

Success Metrics for your Roadmap

Phase	Milestone	Proof of Success
Week 1	Autograd works.	<code>loss.backward()</code> populates non-zero gradients.
Week 2	Sparsity works.	You can set 50% of weights to 0 and the model still predicts correctly.
Week 3	API works.	You can trigger a training session via a <code>curl</code> command.